

ESCUELA DE INGENIERÍA INFORMÁTICA DE OVIEDO

Arquitectura de Computadores

Curso 2025-2026

Teamwork

Díaz Mendaña, Diego - UO301887

García Pernas, Pablo - UO300167

Rama García, Mateo - UO

Suárez Fernández, Fernando - UO300028

Grupo de prácticas: PL.1 - B

Titulación: PCEO Informática y Matemáticas

26 de septiembre de 2025

Índice

1. Introducción	2
2. Desarrollo del Proyecto	2
3. Resultados y Análisis	2
4. Conclusiones	2
A. Flujo de trabajo y metodología de desarrollo	3
A.1. Organización del equipo	3
A.2. Herramientas de desarrollo colaborativo	3
A.2.1. Control de versiones	3
A.2.2. Gestión de tareas	3
A.3. Proceso de desarrollo	4
A.3.1. Workflow de código	4
A.3.2. Estándares de calidad	4
B. Configuración del entorno de desarrollo	4
B.1. Especificaciones de la máquina virtual	4
B.2. Software preinstalado	4
B.3. Configuración adicional	5
B.3.1. Guest Additions	5
B.3.2. Variables de entorno	5
C. Dataset de imágenes y recursos	5
C.1. Estructura del dataset	5
C.2. Características técnicas	5
D. Herramientas de análisis y profiling	5
D.1. Herramientas de medición de rendimiento	5
D.1.1. Timing y benchmarking	5
D.1.2. Profiling de memoria	6
D.1.3. Análisis de rendimiento del procesador	6
D.2. Comandos típicos de análisis	6
E. Plantillas y estándares de documentación	6
E.1. Plantillas de GitHub	6
E.1.1. Plantilla de Pull Request	6
E.1.2. Plantillas de Issues	7
E.2. Convenciones de código	7
E.2.1. Estilo de código C	7
E.2.2. Convenciones de commits	7

F. Métricas y KPIs del proyecto	7
F.1. Métricas de desarrollo	7
F.2. Métricas de rendimiento objetivo	7
G. Referencias técnicas y bibliografía	7
G.1. Documentación oficial	7
G.2. Recursos de optimización	8
G.3. Herramientas de desarrollo	8

1. Introducción

Esta memoria recoge el desarrollo del proyecto de la asignatura *Arquitectura de Computadores*, impartida en el curso 2025-2026 en la Escuela de Ingeniería Informática de Oviedo. El proyecto tiene como objetivo aplicar los conocimientos adquiridos en la asignatura para diseñar e implementar un filtro de imágenes en lenguaje C/C++. Además, se ha de analizar el rendimiento del filtro y compararlo las implementaciones monohilo y multihilo.

El trabajo se ha estructurado en varias fases progresivas:

- **Entrega 1 (Monohilo):** Implementación básica del filtro de imágenes en un solo hilo.
- **Entrega 2 (SIMD y Multihilo):** Optimización del filtro utilizando instrucciones SIMD y paralelización con múltiples hilos.

2. Desarrollo del Proyecto

....

3. Resultados y Análisis

....

4. Conclusiones

....

A. Flujo de trabajo y metodología de desarrollo

Este anexo describe la metodología de desarrollo colaborativo empleada durante el proyecto, incluyendo las herramientas utilizadas, el flujo de trabajo con Git y GitHub, y las buenas prácticas implementadas para garantizar la calidad del código.

A.1. Organización del equipo

El equipo se organizó siguiendo una metodología ágil adaptada, con roles específicos para cada miembro:

- **Project Lead & Backend Developer:** Diego Díaz Mendaña - Coordinación general y desarrollo core
- **SIMD Specialist & Performance Analyst:** Pablo García Pernas - Optimización vectorial
- **Infrastructure & Testing Lead:** Mateo Rama García - Setup de entorno y testing
- **Documentation & QA Manager:** Fernando Suárez Fernández - Documentación y calidad

A.2. Herramientas de desarrollo colaborativo

A.2.1. Control de versiones

Se utilizó **Git** como sistema de control de versiones, con **GitHub** como plataforma de colaboración. La estrategia de branching siguió un modelo adaptado de Git Flow:

- **main:** Rama principal con código estable
- **feature/*:** Ramas para nuevas funcionalidades
- **docs/*:** Ramas específicas para documentación
- **infra/*:** Ramas para configuración e infraestructura

A.2.2. Gestión de tareas

Todas las tareas se gestionaron mediante **GitHub Issues** utilizando plantillas específicas:

- Plantillas para código, documentación, testing e infraestructura
- Sistema Kanban con GitHub Projects (To Do → In Progress → Review → Done)
- Etiquetado por prioridad, tipo y complejidad

A.3. Proceso de desarrollo

A.3.1. Workflow de código

1. Creación de issue con plantilla correspondiente
2. Asignación y creación de rama `feature/nombre-tarea`
3. Desarrollo con commits atómicos y descriptivos
4. Pull Request con plantilla obligatoria
5. Code Review por al menos un compañero
6. Merge tras aprobación y tests exitosos

A.3.2. Estándares de calidad

- **Code Review obligatorio:** Revisión por pares antes de merge
- **Testing sistemático:** Tests unitarios y de rendimiento
- **Documentación continua:** Actualización en cada cambio
- **Convenciones de código:** Estilo consistente y comentarios descriptivos

B. Configuración del entorno de desarrollo

B.1. Especificaciones de la máquina virtual

El desarrollo se realizó en una máquina virtual con las siguientes características:

Componente	Especificación
Sistema Operativo	Ubuntu Desktop 18.04.3 LTS
Usuario	student
Contraseña	teamwork
CPU	Máximo núcleos disponibles (mín. 2)
RAM	Mínimo 3 GiB
Almacenamiento	20+ GB libres

Cuadro 1: Especificaciones de la VM de desarrollo

B.2. Software preinstalado

- `build-essential`: Herramientas de compilación (GCC, make, etc.)
- `git`: Control de versiones
- `cimg-dev`: Librería CImg para procesamiento de imágenes
- `openssh-server`: Servidor SSH para acceso remoto
- **Visual Studio Code** con extensiones C/C++ y TODO Highlight

B.3. Configuración adicional

B.3.1. Guest Additions

Instalación de VirtualBox Guest Additions para mejor integración:

```
1 sudo sh /media/$USER/VBox*/VBoxLinuxAdditions.run
2 sudo reboot
```

B.3.2. Variables de entorno

```
1 export PROJECT_ROOT="$HOME/Desktop/teamwork-ac-2025"
2 export IMAGES_PATH="$HOME/images"
3 export CC=gcc
4 export CFLAGS="-Wall -Wextra -std=c99"
```

C. Dataset de imágenes y recursos

C.1. Estructura del dataset

El dataset oficial proporcionado para el proyecto se organizó en las siguientes categorías:

Directorio	Descripción
normal/	Imágenes con exposición correcta para testing básico
contrast/	Imágenes con alteración de contraste para casos extremos
whitebalance/	Imágenes con alteración de balance de blancos
backgrounds/	Fondos degradados para pruebas de fusión

Cuadro 2: Estructura del dataset de imágenes

C.2. Características técnicas

- **Formatos soportados:** JPG, PNG, BMP
- **Resoluciones típicas:** 1920×1080 , 1280×720 , 800×600
- **Profundidad de color:** 24-bit RGB
- **Tamaño total:** Aproximadamente 50MB

D. Herramientas de análisis y profiling

D.1. Herramientas de medición de rendimiento

D.1.1. Timing y benchmarking

- `gettimeofday()`: Medición de alta precisión
- `clock_gettime()`: Timing monotónico

- **Scripts personalizados:** Automatización de benchmarks

D.1.2. Profiling de memoria

- **Valgrind (massif):** Análisis de uso de memoria
- **Valgrind (memcheck):** Detección de memory leaks
- **/proc/self/status:** Monitoreo en tiempo real

D.1.3. Análisis de rendimiento del procesador

- **perf stat:** Estadísticas de hardware counters
- **perf record/report:** Profiling detallado
- **gprof:** Profiling tradicional de funciones

D.2. Comandos típicos de análisis

```
1 # Benchmarking básico
2 time ./release/programa input.jpg output.jpg
3
4 # Análisis con perf
5 perf stat -e cycles,instructions,cache-misses ./release/programa
6
7 # Memory profiling
8 valgrind --tool=massif ./release/programa input.jpg
9
10 # CPU profiling
11 perf record -g ./release/programa
12 perf report
```

E. Plantillas y estándares de documentación

E.1. Plantillas de GitHub

El proyecto implementó plantillas estandarizadas para mantener consistencia en la documentación:

E.1.1. Plantilla de Pull Request

Incluye secciones para:

- Descripción detallada de cambios
- Tipo de cambio y criterios de aceptación
- Lista de verificación de calidad
- Evidencias de testing y benchmarking
- Instrucciones para revisores

E.1.2. Plantillas de Issues

- **Code Task:** Para implementación de algoritmos
- **Testing Task:** Para análisis de rendimiento y validación
- **Documentation Task:** Para memoria y documentación técnica
- **Infrastructure Task:** Para setup y configuración
- **Bug Report:** Para reporte estructurado de errores

E.2. Convenciones de código

E.2.1. Estilo de código C

- **Estándar:** C99
- **Indentación:** 4 espacios
- **Nomenclatura:** snake_case para funciones y variables
- **Comentarios:** Descriptivos para lógica compleja

E.2.2. Convenciones de commits

Seguimiento de Conventional Commits:

```
1 feat(filtros): implementar filtro de mediana
2 fix(memory): corregir memory leak en liberación
3 docs(readme): actualizar guía de instalación
4 perf(simd): optimizar bucle principal con AVX2
```

F. Métricas y KPIs del proyecto

F.1. Métricas de desarrollo

El proyecto estableció métricas específicas para evaluar el progreso y la calidad:

Métrica	Objetivo	Resultado
Code Coverage	>80 %	[Por completar]
PR Review Time	<24h	[Por completar]
Issue Resolution	<3 días	[Por completar]
Build Success Rate	>95 %	[Por completar]

Cuadro 3: KPIs de calidad del desarrollo

F.2. Métricas de rendimiento objetivo

G. Referencias técnicas y bibliografía

G.1. Documentación oficial

- Intel Intrinsic Guide: <https://software.intel.com/sites/landingpage/IntrinsicsGuide/>

Versión	Tiempo objetivo	Speedup	Memoria
Monohilo	10-15s	1.0× (baseline)	<512MB
SIMD	3-5s	3-4×	<512MB
Multihilo	2-4s	4-8×	<1GB

Cuadro 4: Objetivos de rendimiento por fase

- CImg Library Documentation: <http://cimg.eu/>
- GNU Make Manual: <https://www.gnu.org/software/make/manual/>

G.2. Recursos de optimización

- Tommesani, T. “MMX and SSE Primer”: <https://www.tommesani.com/MMXPrimer.html>
- CodeProject. “Crunching Numbers with AVX and AVX2”: <https://www.codeproject.com/Articles/874396/Crunching-Numbers-with-AVX-and-AVX>
- Finis, J. “x86 Intrinsics Cheat Sheet v2.2”: <https://db.in.tum.de/~finis/x86-intrin-cheatsheet2.pdf>

G.3. Herramientas de desarrollo

- Valgrind User Manual: <http://valgrind.org/docs/manual/>
- Linux perf Wiki: <https://perf.wiki.kernel.org/>
- GDB User Manual: <https://www.gnu.org/software/gdb/documentation/>